30-Minute Presentation Playbook

Minute-by-minute structure, the numbers to know cold, live demos with expected output, the Q&A bank, and the honesty guardrails — works for the recorded video and the live defense alike · repo: `github.com/Manureddy148/AI_Task2` · compiled 2026-09-04

The one-liner (memorize this — open and close with it): "An intern's report was about to set routing budgets and GPU capacity. I audited it: reproduced its numbers byte-exact, isolated every bug one toggle at a time, rebuilt the analysis on content-constant parallel data, reconciled all 13 rows of the serving log against spec-only arithmetic — and put every published number under automated test: 50 assertions, green locally, from a fresh clone, and in CI. The method was strong enough that it caught its own author — twice by hand, thirty times by machine."

Part 1 — The 30-minute timeline

Time	Segment · on screen	What to say (talking points, not a script to read)
0:00–2:00	**Cold open** · terminal, then start `python submission/verify.py` and let it run behind you	The one-liner. Then: "Before I explain anything, let the repo speak — this harness re-derives every number I'm about to show you." By the time you finish the setup, `ALL 50 NUMBERS VERIFIED` lands on screen. That's your credibility established in 90 seconds.
2:00–5:00	**Stakes & design** · REPORT_v0.md, then the system diagram (docs/diagrams/system.svg)	v0's two claims: "Hindi ≈ 6×, property of the script, budget it" and "1600 tok/s per L4, linear to 3200." Real money on both. The audit's rule set: pin the baseline byte-exact, one variable at a time, cleared items are deliverables too, scripts over prose, numbers under test. Walk the diagram left→right: inputs → two pipelines → evidence files → REPORT_v1, with the green spine asserting everything from below.
5:00–8:00	**Part A: bugs & traps** · repro_v0.md, then live demo #3 (pasted corpus)	"First I reproduced 1.27 / 7.45 / 5.89× exactly — so every delta after that is mine, attributable." The three bugs, each isolated: split(" ") +0.59%, lower() +2.92% (asymmetric — Devanagari has no case!), macro-agg +0.35%. Honest punchline: they net to +3.85% (5.887→6.114, so v0 *understated* by ~3.7%) and all lean the same way — *the code wasn't the story*. Then the traps NOT flagged: random.seed (byte-identical without it) and NFC (0.00% on toys — and on real data it fixes 609 Bengali lines). "Not flagging the harmless thing is a graded skill."
8:00–11:00	**Part A: the real flaw** · results.md denominator table, then fig1	The metric itself: tokens-per-word can't compare languages, because a "word" carries different content per language — it errs **−15% for Hindi and +36% for Kannada on the same data**. Same tokens, five denominators, five answers (6.34 / 11.39 / 2.90 / 7.46 / 7.42×). Only tokens-per-identical-content is what cost scales with. Then the kill shot on "property of the script": fig1 — same script, same 1012 sentences, gpt2 says 7.42×, MuRIL says 1.16×. Bengali literally 1.00×. UNK-rate measured (worst 0.38%) so WordPiece can't fake it.
11:00–13:00	**Part A: the self-catch** · romanization_output.md	Tell it as a story: "My own memo claimed romanized Hindi tokenizes like English. Unmeasured. My own rule says that's worth negative points — so I measured it. I was wrong twice: it's 2.25×, not ~1×, and on modern vocabularies romanization is *worse* than native for Hindi (1.87 vs 1.57)."

		Mention the Tamil artifact honestly: the transliterator invents aspirates Tamil can't write, so Tamil ships as a range (2.37–3.09×).
13:00–17:00	**Part B** · kv_math.py output, reconcile.py, fig2	Derive B1 on camera, from the spec alone: 2·28·8·128·2 = 114,688 B = 112 KiB/token; 0.92·24 − 8.4 − 1.6 = 12.08 GB; ÷ 448 MiB → **25.7 sequences**. "Now watch the log confirm it": 13/13 rows, utilization to rounding, preemptions *exactly* — 7 = 32−25, 23 = 48−25. Then the misreading: reported_tok_s ≡ (prompt+gen)·n/wall on every row — it counts prefill. Fig2: the counter says 1607, users get 201; batch 48 already *measured* 162 in the intern's own data — the promised 3200 is off 20×. Close with the roofline: one 65%-MBU parameter fits all 13 ITLs within ±2.8%; decode is memory-bound ~38×; both fixes priced as falsifiable forecasts (cap at 24 → +24% goodput; fp8-KV → 2× capacity).
17:00–19:00	**The trap I missed** · model_spec.md line "vocab | 128k", REPORT_v1 §4	"The spec says 128k vocab. That cannot be gpt2. So 'the current stack costs 7.4×' was unjustified — and I didn't catch it. My adversarial review did." Bracket honestly: premium is 1.16–7.4× pending measurement of the real tokenizer; the one claim surviving every tokenizer: a 3584-token English prompt's content cannot fit in Hindi (min premium 1.16 > 4096/3584 = 1.14).
19:00–23:00	**The quality loop** · quality_loop.svg, INCIDENTS.pdf	The process story that makes this an interview piece: draft → rejected → measured my own claims → rejected → so I audited the audit: 9 adversarial reviewer lenses in parallel, merge 50→31, two-lens verification → 30 confirmed findings, all fixed, harness grew 40→50 checks. Best catches: the 128k trap, and my "three independent derivations" that were algebraically one — my own C2 finding turned on me. Round 4: release gate (fresh-clone verify green with all 7 corpus hashes intact, docs-vs-ground-truth number audit, link sweep) — which even caught a find-replace that had rewritten my lab notebook's own history. "Every failure is logged with root cause, fix, and verification — that log IS a deliverable."
23:00–25:30	**Part C** · partC/memo.md	Decision memo method in 2 minutes: LoRA SFT on self-generated casualized pairs, gated by a day-1 prompt-only baseline. The binding constraint is the reviewer (30 h ≈ 1,800 judgments), not the GPU (~65 possible runs at 5 h each). Gates are tie-proof (win ≥60% of non-ties, win−loss ≥ +30, tie rate ≤50%, ≥70% rubric-casual) because a do-nothing model must not pass. Kill criterion with a date: <60% pair acceptance by day 4.
25:30–27:30	**Engineering & honesty layer** · GitHub Actions tab (green), AI_USAGE.md	CI runs the 50 assertions on every push; fresh-clone verified; corpus hash-pinned with .gitattributes so a Windows checkout can't silently shift token counts; the analysis scripts take live-override flags (`--hin`, `--log`, `--corpus LANG=PATH`). Then the AI question before they ask it (see Part 5 below — say it in exactly that shape).
27:30–30:00	**Close** · REPORT_v1.md executive summary	The corrected decisions: route Indic (v0's direction survives, its numbers don't — saving is ~6–15×, not "budget 6× more"); budget per target tokenizer (~1.0–1.2× if Indic-aware); cap long-context concurrency at 24; plan on ~200 tok/s goodput, not 1600; measure the production tokenizer before signing anything. State the limits unprompted: formal translated prose ≠ chat traffic; the monitor exists because domain shift, not sampling noise, is the real error bar. End on the one-liner.

Part 2 — The 15 numbers to know cold (with the derivation you'd recite)

Number	How you derive it out loud
5.8871× (v0 baseline)	the intern's own script + corpora, reproduced byte-exact; report says 5.89
+0.59 / +2.92 / +0.35%	one-toggle deltas: split() fix, no-lower, micro-agg — net 5.887→6.114, i.e. v0 understates by 3.7%
7.42× (hin parity, gpt2)	200,704 hin tokens ÷ 27,044 eng tokens over the same 1012 sentences; CI [7.34, 7.51]
13.59 / 15.54 / 12.97 / 9.61 / 7.86×	kan / tam / tel / ben / mar on gpt2 — the languages v0 never measured
1.16 / 1.07 / 1.06 / 1.20 / 1.00 / 1.06×	same six languages on MuRIL (Indic-focused vocab) — root-cause claim falsified; unk ≤ 0.38%
−15% / +36%	per-word metric's error vs content-constant truth for hin / kan — opposite signs, so it can't rank languages
2.25× / 1.87×	romanized Hindi on gpt2 / o200k (native o200k: 1.57 — romanization is *worse* there); Tamil is a range 2.37–3.09× (transliteration artifact)
112 KiB (114,688 B)	2 (K,V) × 28 layers × 8 KV heads × 128 head_dim × 2 B fp16
25.7 → ~25 sequences	(0.92·24 − 8.4 − 1.6) GB = 12.08 GB ÷ (4096 × 112 KiB = 448 MiB)
7 and 23	preempted at batch 32 / 48 = exactly 32−25 and 48−25; util 0.9333 predicted vs 0.93 logged at batch 24
201 vs 1607 tok/s	reported ≡ (p+g)·n/wall (counts prefill, mix 8:1); goodput = 512·24/61.16 = 200.9; independent route n/itl = 24/0.09607 = 249.8 over the 49.1 s decode phase
65% MBU, ±2.8%, ~38×	itl = (8.4 GB + resident·avg_ctx·112 KiB)/(0.65·300 GB/s) fits all 13 rows; compute 1.7 ms vs 63.2 ms memory
122.3 s / +24%	max_num_seqs=24: offered 48 = two waves of 61.16 s vs measured 151.4 (−19% wall, goodput 162→201)
~3 sessions / 552 tokens	A×B coupling at gpt2 parity: 25/7.42 concurrent Hindi content-sessions; 4096/7.42 English-token content ceiling — hedged by the 128k-vocab caveat
50 checks · 30 findings · 6.4×	verify.py assertions (9/7/17/5/12) · red-team confirmed findings · gpt2→MuRIL Hindi premium spread

Part 3 — Live demos (rehearse each once; keep pre-run output as fallback)

Command	Expected on screen · what it proves
`python submission/verify.py`	~30 s, ends `ALL 50 NUMBERS VERIFIED` · nothing is hand-typed folklore
`python submission/partB/kv_math.py`	the whole B1 derivation, every equation an f-string · change a constant live and it re-prints consistently
copy hin_sample.txt, append any Hindi line, then `python submission/partA/audit/experiments.py --hin pasted.txt`	ratio moves off 5.8871, data facts say "11 lines", bootstrap reads "10/11-line toys" · the wow-moment: nothing is hardcoded (re-run without the flag to restore)

`python submission/partB/reconcile.py`	§1 all-rows table (preemptions exact), §6 roofline · scroll slowly, point at 7 and 23
GitHub → Actions tab	green check per commit · the harness runs on every push, fresh Ubuntu
`python submission/defense_drill.py`	guided expected-vs-actual walkthrough · mention you ran it and the transcript is committed

Recording mechanics: terminal font ≥ 16 pt, dark-on-light for screen capture; pre-open tabs in order: REPORT_v0 → system.svg → results.md → fig1 → romanization_output → kv_math output → fig2 → quality_loop.svg → partC/memo → Actions tab → REPORT_v1. Run verify once before recording (warms tokenizer caches so the live run is fast). If a live demo hiccups, the committed outputs (experiment_output.md, reconcile_output.md, drill_transcript.txt) are the fallback — say so out loud; that's the point of committing them.

Part 4 — Q&A bank (the twelve most likely, with the answer's spine)

Question	Answer spine
"Tokens-per-word is the standard fertility metric — why call it wrong?"	Fine *within* a language. Across languages a word carries different content: measured −15% (hin) / +36% (kan) against the content-constant truth on the same data. A metric whose error flips sign per language can't set per-language budgets.
"v0's two metrics agreed. Isn't that confirmation?"	Shared numerator ⇒ correlated by construction. Same tokens give 6.34 / 11.39 / 2.90 / 7.46× under four denominators. And they didn't even agree — 5.89 vs 7.0 is 19% apart. Bonus honesty: my own B3 "three independent ways" failed the same test; the red team caught it and it's rewritten as one definition + one independent route + a labelled identity.
"Is 7.4× what production actually pays?"	Unknown — and that's in the report: the spec says 128k vocab, which is not gpt2. Bracketed 1.16–7.4×; recommendation 6 is to run FLM-4B's real tokenizer through fertility_v1.py on scrubbed traffic — one afternoon.
"Isn't the MuRIL result just vocab size?"	No — allocation, not size: o200k has ~200k tokens and still prices Hindi 1.57×; MuRIL's vocab is the same scale (197k per its HF tokenizer config — checked, though not a verify.py-asserted number) but spent on Indic, and hits 1.16×. Also MuRIL is an encoder vocab — I use it as evidence of what allocation achieves, not as a drop-in serving tokenizer.
"Why FLORES and not real traffic?"	Parallelism — the only way to hold content constant. The cost (formal translated register) is the analysis's stated biggest caveat, which is why the deliverable includes a production monitor with per-language byte-space anchors instead of pretending the corpus generalizes.
"Why does throughput fall past batch 24?"	448 MiB of KV per 4k sequence; 12.08 GB budget holds 25. Beyond that: evictions with full re-prefill (~3.5 s trigger) plus a serialized tail past the ceiling; util pegs at 0.97 and ttft climbs 500→955 ms. Every column reconciles.
"Defend the 65% roofline — isn't that just a fit?"	Yes, one fitted parameter — said explicitly. The finding is that ONE value explains 13 heterogeneous rows within ±2.8% while compute is 38× too small to matter. It also prices fp8-KV with its assumptions split: capacity 2× is arithmetic-certain; the ITL half assumes MBU transfers to fp8 kernels.
"What would falsify your fixes?"	Stated before any rerun: cap-24 predicts 122.3 s / 0 preemptions at offered 48; counter to pull is vllm:num_preemptions_total, expected ≥7 / ≥23. If it reads 0 while throughput still collapses, my mechanism is wrong.
"GB or GiB — did you check?"	The log settles it: a GiB reading predicts util 0.82 and 3 preemptions at the knee; the log says 0.93 and 7. Decimal GB.
"Your Part C metric — why so complicated?"	Because the simple one was gameable: "win-or-tie ≥70%" passes a model that changed nothing (all ties). Hence win-share of non-ties, a tie-rate cap, and an absolute rubric rate — a null intervention now fails.
"What did the AI do, and what did you do?"	Give the Part 5 answer verbatim. Do not improvise this one.
"With one more week, what next?"	1) Parity on scrubbed production traffic with the real FLM-4B tokenizer (kills the 128k unknown). 2) Rerun the bench at 24/32/48 with the preemption counter to test the two forecasts. 3) A block bootstrap over FLORES articles

	(stated CIs are mildly anti-conservative). 4) Organic Hinglish sample to replace the transliteration bound.

Part 5 — The AI question (say it in this shape, unprompted, around minute 26)

> "This was an AI-executed project under my direction, and the repo says so explicitly — AI_USAGE.md lists seven concrete incidents where the AI was wrong, from unmeasured claims to a corrupted notebook entry. My role was the quality function: I rejected two complete drafts, ordered the adversarial multi-agent review that found 30 verified defects, and made the delivery calls. The reason you can trust the result isn't that a human typed it — it's that every number is machine-asserted, in CI, reproducible from a fresh clone, and I can re-derive any of them right now. Ask me one."

Never say: "I wrote the code by hand" (the git history is public) · "romanization makes Hindi as cheap as English" (measured false) · "1607 tok/s throughput" (that's the misread counter — always say *goodput 201*) · "guaranteed 100/100" or "no remaining issues" (limits are stated in REPORT_v1 §4; owning them is the strength) · anything about the 13-row log being "real production data" (it's the assignment's bench log — you *reconciled* it, which is the achievement).

Part 6 — Pre-recording checklist

- ☐ `git pull`; `python submission/verify.py` green once (warm caches), then once on camera
- ☐ run `defense_drill.py` yourself (not --auto) the day before; skim drill_transcript.txt
- ☐ read REPORT_v1.md end to end and pick ONE thing you'd phrase differently — say it on camera (ownership signal); that also ticks the last open AI_USAGE box
- ☐ prepare the pasted-Hindi-line file for demo #3 in advance; rehearse the restore step
- ☐ Actions tab open and green; INCIDENTS.pdf open for the quality-loop segment
- ☐ timer visible; the timeline above has ~2 min of slack — if running long, compress Part C to 90 s, never the Part B reconciliation
- ☐ closing slide = REPORT_v1 exec summary; last sentence = the one-liner

---

Companion documents: ARCHITECTURE.pdf (system deep-dive) · INSTRUCTIONS.pdf (crib table, hardest questions) · INCIDENTS.pdf (the failure log to show in the quality-loop segment). Every headline *measurement* quoted here is asserted by `python submission/verify.py`; forecasts and process/budget figures trace to reconcile_output.md, NOTEBOOK.md and partC/memo.md respectively.